

Universidad Autónoma de Chiapas

TEMA: Confidencialidad

Documentación Técnica: Implementación de API REST y  
Seguridad

Ingeniería en Desarrollo de Software

5 de octubre de 2026

# Índice

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                                | <b>3</b> |
| <b>2. 1. Requerimientos Funcionales</b>                               | <b>3</b> |
| 2.1. 1.1 Gestión de Usuarios . . . . .                                | 3        |
| 2.1.1. Registro de Usuarios . . . . .                                 | 3        |
| 2.1.2. Inicio de Sesión y Recuperación . . . . .                      | 4        |
| 2.2. 1.2 Gestión de Roles y Permisos . . . . .                        | 4        |
| 2.3. 1.3 Control de Acceso y Dashboard de Administración . . . . .    | 5        |
| <b>3. 2. Arquitectura Web y Consumo de Servicios</b>                  | <b>5</b> |
| <b>4. 3. Requerimientos de Seguridad Computacional</b>                | <b>6</b> |
| 4.1. 3.1 Cifrado de Contraseñas . . . . .                             | 6        |
| 4.2. 3.2 Comunicación Cifrada (HTTPS/TLS) . . . . .                   | 6        |
| 4.3. 3.3 Gestión Segura de Tokens JWT . . . . .                       | 6        |
| 4.4. 3.4 Autorización Verificada en el Servidor . . . . .             | 7        |
| 4.5. 3.5 Validación y Sanitización de Entradas (XSS y SQLi) . . . . . | 7        |
| 4.6. 3.6 Protección contra Ataques de Fuerza Bruta . . . . .          | 7        |
| 4.7. 3.7 Registro de Auditoría con Trazabilidad . . . . .             | 8        |
| 4.8. 3.8 Manejo Seguro de CORS y Fuga de Información . . . . .        | 8        |
| <b>5. Conclusión</b>                                                  | <b>8</b> |

## 1. Objetivo del Caso Práctico

El objetivo es desarrollar una aplicación web que implemente autenticación y autorización basada en roles (RBAC), consumiendo una API REST protegida mediante JSON Web Tokens (JWT). Se garantiza que los usuarios accedan a diferentes funcionalidades según su rol, asegurando la confidencialidad e integridad de la información en todo momento.

**Solución Tecnológica:** Se eligió **Laravel 11** (PHP) como motor backend y un frontend desacoplado desarrollado en HTML/CSS/JS (Vanilla).

## 2. Requerimientos Funcionales

### 2.1. 1. Gestión de Usuarios

Se crearon los endpoints en `AuthController.php`:

- **Registro:** POST `/api/v1/register` (Nombre, Email, Contraseña).
- **Inicio de Sesión:** POST `/api/v1/login` que devuelve el JWT.
- **Recuperación de Contraseña:** La arquitectura de base de datos soporta los flujos de `password_resets` integrados en Laravel.

### 2.2. 2. Gestión de Roles

Se integró el paquete **Spatie Laravel Permission** para gestionar de manera dinámica los tres roles predeterminados:

- **Administrador:** Tiene acceso total (Lectura, Escritura, Eliminación).
- **Editor:** Tiene permisos limitados (Lectura, Escritura).
- **Usuario Regular:** Solo tiene permiso de lectura.

Estos fueron insertados en la base de datos a través de `RolesAndPermissionsSeeder.php`.

### 2.3. 3. Gestión de Permisos

Los permisos (*Lectura*, *Escritura*, *Eliminación*) están desacoplados del código fuente y residen en tablas relacionales (`role_has_permissions`). Esto permite asignar permisos a los roles de forma dinámica y aplicar políticas a áreas específicas de la aplicación.

### 2.4. 4. Control de Acceso y Auditoría

El acceso está controlado desde el backend usando `Route::middleware('role:Administrador')`. Si un Editor intenta acceder a una ruta de Administrador, el servidor responde con **403 Forbidden**.

**Historial de Acceso:** Se creó el modelo `AuditLog` en Laravel. En cada acción (registro, login, asignar rol), se captura la dirección IP del usuario, ID, y acción, guardándola de forma inmutable.

## 2.5. 5. Dashboard de Administración

El `AdminController.php` expone las siguientes rutas protegidas exclusivas para administradores:

- Ver usuarios registrados (`GET /admin/users`).
- Crear, editar y eliminar usuarios (`POST`, `PUT`, `DELETE`).
- Asignar/revocar roles (`POST /admin/users/{id}/role`).
- Crear roles (`POST /admin/roles`).
- Consultar registros de auditoría (`GET /admin/audit-logs`).

## 2.6. 6. Arquitectura Web y Consumo de Servicios

Se aplicó una **arquitectura desacoplada estricta**:

- **Frontend:** Es un cliente tonto (`public/admin.html`) que no tiene acceso a la base de datos.
- **Backend:** Expone todo exclusivamente a través de la API REST (`routes/api.php`).
- **JWT:** La comunicación viaja en los headers como `Authorization: Bearer <token>`. No se usan Vistas de Blade para el renderizado del dashboard para no romper la regla del estado (Stateless).

## 3. Requerimientos Técnicos

1. **Lenguaje y SMD:** PHP con Laravel 11. Motor de base de datos MySQL/SQLite a través del ORM Eloquent.
2. **Frontend Web:** Aplicación accesible desde navegador usando HTML/CSS/JS avanzado con diseño Glassmorphism e inyecciones vía AJAX.
3. **API REST Plena:** Se utilizaron los verbos HTTP `GET`, `POST`, `PUT`, `DELETE`. Todas las respuestas devuelven `application/json`.
4. **Autenticación JWT:** El inicio de sesión usa **Laravel Passport** (OAuth2). Valida credenciales, firma el JWT y el middleware `auth:api` se encarga de interceptar y validar automáticamente la firma en peticiones subsecuentes.

## 4. Requerimientos de Seguridad Computacional

### 4.1. 1. Cifrado de contraseñas

Se utilizó el algoritmo **Bcrypt** con *salt* autogenerado utilizando el `Facade Hash::make($password)` de Laravel. Ninguna contraseña es reversible ni se almacena en texto plano.

## 4.2. 2. Comunicación Cifrada (HTTPS/TLS)

Se estableció en las directivas de despliegue que todo el tráfico se forzará por HTTPS. La API interceptará el protocolo para evitar secuestros de sesión (Man-in-the-Middle).

## 4.3. 3. Gestión segura de tokens JWT

- **Expiración:** Originalmente se configuró un tiempo corto. Posteriormente se ajustó para propósitos de evaluación.
- **Firma:** El token es firmado internamente usando claves RSA (RS256) generadas con el comando `passport:keys`. Dichas claves (`oauth-private.key`) tienen permisos estrictos 600 y no se almacenan en el control de versiones.

## 4.4. 4. Autorización verificada en el servidor

Independientemente de que el frontend oculte visualmente botones mediante CSS/JS, la seguridad principal recae en el servidor mediante los Middlewares `role` y `permission`.

## 4.5. 5. Validación y Sanitización de Entradas

Laravel protege contra Inyección SQL mediante sentencias preparadas de PDO en su ORM. Para ataques XSS y validaciones masivas, se usó `$request->validate()` para sanear tipos de datos en todos los Controladores.

## 4.6. 6. Protección contra ataques de fuerza bruta

Se protegió el endpoint `/login` con el middleware `throttle:5,1`. Si un atacante falla la contraseña más de 5 veces en un minuto, recibe un código **429 Too Many Requests**.

## 4.7. 7. Registro de auditoría con trazabilidad

El modelo `AuditLog` guarda el `user_id`, `ip_address`, `action` y `timestamp`. Estos registros solo pueden insertarse mediante lógica en el backend y no pueden ser alterados por nadie, manteniendo su peso legal y confidencial.

## 4.8. 8. Manejo seguro de CORS

El archivo `config/cors.php` restringe qué orígenes pueden consumir la API, previniendo que scripts maliciosos alojados en otros dominios roben la sesión o datos.

## 4.9. 9. Principio de Mínimo Privilegio

Un editor no puede ver la tabla de usuarios. Un usuario no puede borrar contenido. Esto se logró definiendo reglas atómicas en Spatie y limitando los accesos a lo que estrictamente requiere cada función.

## 4.10. 10. Manejo de errores sin fuga de información

Se configuró el entorno `APP_ENV=production` y `APP_DEBUG=false`. En caso de que falle la base de datos o el código, el usuario no verá trazas de pila (Stack Traces) ni configuraciones internas; solo recibirá un error **HTTP 500** genérico.